

Five lectures on the MatterMost paper:

- The theory behind the MatterMost paper (four lectures)
- The main idea of the MatterMost paper (one lecture)

Five lectures on the MatterMost paper:

- The theory behind the MatterMost paper (four lectures)
- The main idea of the MatterMost paper (one lecture)

Today:

- The theory behind the MatterMost paper 4

Wrap up MM3 lecture

Wrap up MM3 lecture

- We have formulated the problem of solving a SDP for n steps starting at decision step t as that of finding a policy sequence (for n steps starting at t) that maximizes a measure of the $\oplus$ -sum of the rewards along all possible trajectories.

Wrap up MM3 lecture

- We have formulated the problem of solving a SDP for n steps starting at decision step t as that of finding a policy sequence (for n steps starting at t) that maximizes a measure of the $\oplus$ -sum of the rewards along all possible trajectories.
- We have formalized this idea through a notion of **optimality** (for policy sequences for n steps starting at t):

$$\text{OptPolicySeq} : \{t\} \times \mathbb{N} \rightarrow \text{PolicySeq } t\ n \rightarrow \text{Set}$$

$$\text{OptPolicySeq } \{t\} \{n\} \text{ ps} = \forall (ps' : \text{PolicySeq } t\ n) \rightarrow \text{val } ps' \leq_I \text{val } ps$$

Wrap up MM3 lecture

- We have formulated the problem of solving a SDP for n steps starting at decision step t as that of finding a policy sequence (for n steps starting at t) that maximizes a measure of the $\oplus$ -sum of the rewards along all possible trajectories.
- We have formalized this idea through a notion of optimality (for policy sequences for n steps starting at t):

$$\text{OptPolicySeq} : \{t\ n : \mathbb{N}\} \rightarrow \text{PolicySeq } t\ n \rightarrow \text{Set}$$

$$\text{OptPolicySeq } \{t\} \{n\} \text{ } ps = \forall (ps' : \text{PolicySeq } t\ n) \rightarrow \text{val } ps' \leq_I \text{val } ps$$

where

$$_ \leq_I _ : \{A : \text{Set}\} \rightarrow (A \rightarrow \text{Val}) \rightarrow (A \rightarrow \text{Val}) \rightarrow \text{Set}$$

$$f \leq_I g = \forall x \rightarrow f\ x \leq g\ x$$

$$\text{val} : \{t\ n : \mathbb{N}\} \rightarrow \text{PolicySeq } t\ n \rightarrow X\ t \rightarrow \text{Val}$$

$$\text{val } ps\ x = \text{measure } (\text{fmap}_M \text{sumR } (\text{trj } ps\ x))$$

Solution: Bellman's equation, backward induction

Solution: Bellman's equation

- We have understood **what** it means for a policy sequence to be optimal but ...

Solution: Bellman's equation

- We have understood what it means for a policy sequence to be optimal but ...
- ... **how** can one compute optimal policy sequences?

Solution: Bellman's equation

- We have understood what it means for a policy sequence to be optimal but ...
- ... how can one compute optimal policy sequences?
- For finite X t , Y t x , an obvious approach is brute-force:

Solution: Bellman's equation

- We have understood what it means for a policy sequence to be optimal but ...
- ... how can one compute optimal policy sequences?
- For finite X t , Y t x , an obvious approach is brute-force:
- **First**, generate all policy sequences pss . Then pick up a $ps \in pss$ such that

$$\forall (ps' \in pss) \rightarrow val\ ps' \leq_I val\ ps$$

$\equiv$

$$\forall (ps' \in pss) \rightarrow (measure \circ fmap_M\ sumR \circ trj\ ps') \leq_I (measure \circ fmap_M\ sumR \circ trj\ ps)$$

Solution: Bellman's equation

- We have understood what it means for a policy sequence to be optimal but ...
- ... how can one compute optimal policy sequences?
- For finite X t , Y t x , an obvious approach is brute-force:
- First, generate all policy sequences pss . Then pick up a $ps \in pss$ such that

$$\forall (ps' \in pss) \rightarrow val\ ps' \leq_I val\ ps$$

$\equiv$

$$\forall (ps' \in pss) \rightarrow (measure \circ fmap_M\ sumR \circ trj\ ps') \leq_I (measure \circ fmap_M\ sumR \circ trj\ ps)$$

Solution: Bellman's equation

- We have understood what it means for a policy sequence to be optimal but ...
- ... how can one compute optimal policy sequences?
- For finite X t , Y t x , an obvious approach is brute-force:
- First, generate all policy sequences pss . Then pick up a $ps \in pss$ such that

$$\forall (ps' \in pss) \rightarrow val\ ps' \leq_I val\ ps$$

$\equiv$

$$\forall (ps' \in pss) \rightarrow (measure \circ fmap_M\ sumR \circ trj\ ps') \leq_I (measure \circ fmap_M\ sumR \circ trj\ ps)$$

- Around 1954, Bellman came up with a much better idea: dynamic programming!

Solution: Bellman's equation

- Let's have a look at the value of $[p0, p1]$ in $x_0 : X \rightarrow \mathbb{R}$ for $M = SP$, $SP\ X = List(X, \mathbb{R}_{[0,1]})$ with $Val = \mathbb{R}$, $\oplus = +$, $0_{Val} = 0$ and the expected value measure

Solution: Bellman's equation

- Let's have a look at the value of $[p0, p1]$ in $x_0 : X \rightarrow \mathbb{R}$ for $M = SP$, $SP\ X = List\ (X, \mathbb{R}_{[0,1]})$ with $Val = \mathbb{R}$, $\oplus = +$, $0_{Val} = 0$ and the expected value measure
- Let α and β be arbitrary probabilities and

Solution: Bellman's equation

- Let's have a look at the value of $[p_0, p_1]$ in $x_0 : X \rightarrow \mathbb{R}$ for $M = SP$, $SP(X) = List(X, \mathbb{R}_{[0,1]})$ with $Val = \mathbb{R}$, $\oplus = +$, $0_{Val} = 0$ and the expected value measure
- Let α and β be arbitrary probabilities and

$$p_0 x_0 = y_0 \quad \text{next } 0 x_0 y_0 = [(x_1^0, \alpha), (x_1^1, 1 - \alpha)] \quad \begin{aligned} \text{reward } 0 x_0 y_0 x_1^0 &= r_0^0 \\ \text{reward } 0 x_0 y_0 x_1^1 &= r_0^1 \end{aligned}$$

Solution: Bellman's equation

- Let's have a look at the value of $[p0, p1]$ in $x_0 : X$ for $M = SP$, $SP\ X = List\ (X, \mathbb{R}_{[0,1]})$ with $Val = \mathbb{R}$, $\oplus = +$, $0_{Val} = 0$ and the expected value measure
- Let α and β be arbitrary probabilities and

$$p_0\ x_0 = y_0 \quad next\ 0\ x_0\ y_0 = [(x_1^0, \alpha), (x_1^1, 1 - \alpha)] \quad reward\ 0\ x_0\ y_0\ x_1^0 = r_0^0$$

$$reward\ 0\ x_0\ y_0\ x_1^1 = r_0^1$$

$$p_1\ x_1^0 = y_1^0 \quad next\ 1\ x_1^0\ y_1^0 = [(x_2^{0,0}, \beta), (x_2^{0,1}, 1 - \beta)] \quad reward\ 1\ x_1^0\ y_1^0\ x_2^{0,0} = r_1^{0,0}$$

$$reward\ 1\ x_1^0\ y_1^0\ x_2^{0,1} = r_1^{0,1}$$

Solution: Bellman's equation

- Let's have a look at the value of $[p_0, p_1]$ in $x_0 : X$ for $M = SP$, $SP\ X = List\ (X, \mathbb{R}_{[0,1]})$ with $Val = \mathbb{R}$, $\oplus = +$, $0_{Val} = 0$ and the expected value measure
- Let α and β be arbitrary probabilities and

$$p_0\ x_0 = y_0 \quad next\ 0\ x_0\ y_0 = [(x_1^0, \alpha), (x_1^1, 1 - \alpha)] \quad reward\ 0\ x_0\ y_0\ x_1^0 = r_0^0$$

$$reward\ 0\ x_0\ y_0\ x_1^1 = r_0^1$$

$$p_1\ x_1^0 = y_1^0 \quad next\ 1\ x_1^0\ y_1^0 = [(x_2^{0,0}, \beta), (x_2^{0,1}, 1 - \beta)] \quad reward\ 1\ x_1^0\ y_1^0\ x_2^{0,0} = r_1^{0,0}$$

$$reward\ 1\ x_1^0\ y_1^0\ x_2^{0,1} = r_1^{0,1}$$

$$p_1\ x_1^1 = y_1^1 \quad next\ 1\ x_1^1\ y_1^1 = [(x_2^{1,0}, 1)] \quad reward\ 1\ x_1^1\ y_1^1\ x_2^{1,0} = r_1^{1,0}$$

Solution: Bellman's equation

- Let's have a look at the value of $[p0, p1]$ in $x_0 : X$ for $M = SP$, $SP\ X = List\ (X, \mathbb{R}_{[0,1]})$ with $Val = \mathbb{R}$, $\oplus = +$, $0_{Val} = 0$ and the expected value measure
- Let α and β be arbitrary probabilities and

$$p_0\ x_0 = y_0 \quad next\ 0\ x_0\ y_0 = [(x_1^0, \alpha), (x_1^1, 1 - \alpha)] \quad \begin{aligned} reward\ 0\ x_0\ y_0\ x_1^0 &= r_0^0 \\ reward\ 0\ x_0\ y_0\ x_1^1 &= r_0^1 \end{aligned}$$

$$p_1\ x_1^0 = y_1^0 \quad next\ 1\ x_1^0\ y_1^0 = [(x_2^{0,0}, \beta), (x_2^{0,1}, 1 - \beta)] \quad \begin{aligned} reward\ 1\ x_1^0\ y_1^0\ x_2^{0,0} &= r_1^{0,0} \\ reward\ 1\ x_1^0\ y_1^0\ x_2^{0,1} &= r_1^{0,1} \end{aligned}$$

$$p_1\ x_1^1 = y_1^1 \quad next\ 1\ x_1^1\ y_1^1 = [(x_2^{1,0}, 1)] \quad reward\ 1\ x_1^1\ y_1^1\ x_2^{1,0} = r_1^{1,0}$$

Exercise 7.1

On the fly: How many trajectories are in $trj\ [p0, p1]\ x_0$?

Solution: Bellman's equation

We compute

Solution: Bellman's equation

We compute

$$\begin{aligned} & \text{val } [p0, p1] \ x_0 \\ &= \{ \text{step}_1 \} = \\ & \text{ev } (fmap_{SP} \text{ sumR } (trj [p0, p1] \ x_0)) \\ &= \{ \text{step}_2 \} = \\ & \text{ev } (fmap_{SP} \text{ sumR } [((x_0, y_0) \parallel (x_1^0, y_1^0) \parallel \text{Last } x_2^{0,0}), \alpha * \beta), \\ & \quad ((x_0, y_0) \parallel (x_1^0, y_1^0) \parallel \text{Last } x_2^{0,1}), \alpha * (1 - \beta)), \\ & \quad ((x_0, y_0) \parallel (x_1^1, y_1^1) \parallel \text{Last } x_2^{1,0}), 1 - \alpha]) \\ &= \{ \text{step}_3 \} = \\ & \text{ev } [(r_0^0 + r_1^{0,0}, \alpha * \beta), (r_0^0 + r_1^{0,1}, \alpha * (1 - \beta)), (r_0^1 + r_1^{1,0}, 1 - \alpha)] \\ &= \{ \text{step}_4 \} = \\ & r_0^0 * \alpha + r_1^{0,0} * \beta * \alpha + r_1^{0,1} * (1 - \beta) * \alpha + r_0^1 * (1 - \alpha) + r_1^{1,0} * (1 - \alpha) \end{aligned}$$

Solution: Bellman's equation

Exercise 7.2

In **step₂** we have applied the generic trajectory computation

$$\begin{aligned} \text{trj} &: \{t\ n : \mathbb{N}\} \rightarrow \text{PolicySeq } t\ n \rightarrow X\ t \rightarrow M\ (XYSeq\ t\ (\text{suc } n)) \\ \text{trj } \{t\} \text{ Nil } x &= \eta_M (\text{Last } x) \\ \text{trj } \{t\} (p :: ps) x &= \text{let } y = p\ x \text{ in} \\ &\quad \text{let } mx' = \text{next } t\ x\ y \text{ in} \\ &\quad \text{fmap}_M ((x, y) \parallel _) (mx' \gg=_{\text{M}} \text{trj } ps) \end{aligned}$$

for $M = SP$, $SP\ X = \text{List } (X, \mathbb{R}_{[0,1]})$ and with

$$\begin{aligned} \eta_{SP} x &= (x, 1.0) :: [] \\ \text{fmap}_{SP} f &= \text{fmap}_{List} (\lambda (x, p) \rightarrow (f\ x, p)) \\ sp \gg=_{SP} f &= \mu_{SP} (\text{fmap}_{SP} f\ sp) \end{aligned}$$

Define $\mu_{SP} : SP\ (SP\ A) \rightarrow SP\ A$ in such a way that $\text{trj } [p0, p1] x_0$ yields the result of **step₂**.

Solution: Bellman's equation

- Exercise 7.2: in defining $\mu_{SP}((sp, q) :: sps)$, assume that sp and all the probability distributions in sps are normalized that is, that their probabilities add up to one.

Solution: Bellman's equation

- Exercise 7.2: in defining $\mu_{SP}((sp, q) :: sps)$, assume that sp and all the probability distributions in sps are normalized that is, that their probabilities add up to one.

Exercise 7.3

In $sp \gg=_{SP} f$ with $sp : SP\ A$ and $f : A \rightarrow SP\ B$, the probability of $b : B$ according to $f\ a$, $prob(f\ a)\ b$ can be interpreted as the conditional probability of b given a . The law of total probability en.wikipedia.org/wiki/Law_of_total_probability requires $\gg=_{SP}$ to fulfill a specification. Write this specification in the form

$$\forall b : B. \text{prob}(sp \gg=_{SP} f)\ b = \dots$$

Solution: Bellman's equation

- Exercise 7.2: in defining $\mu_{SP}((sp, q) :: sps)$, assume that sp and all the probability distributions in sps are normalized that is, that their probabilities add up to one.

Exercise 7.3

In $sp \gg=_{SP} f$ with $sp : SP\ A$ and $f : A \rightarrow SP\ B$, the probability of $b : B$ according to $f\ a$, $prob(f\ a)\ b$ can be interpreted as the conditional probability of b given a . The law of total probability en.wikipedia.org/wiki/Law_of_total_probability requires $\gg=_{SP}$ to fulfill a specification. Write this specification in the form

$$\forall b : B. \text{prob}(sp \gg=_{SP} f)\ b = \dots$$

Exercise 7.4

In [step₄](#) we have applied a definition of the exp. value measure [ev](#). Is this consistent with your definition from Exercise 6.3?

Solution: Bellman's equation

$$\begin{aligned} &= \{ \text{step}_5 \} = \\ &r_0^0 * \alpha + r_1^{0,0} * \beta * \alpha + r_1^{0,1} * (1 - \beta) * \alpha + r_0^1 * (1 - \alpha) + r_1^{1,0} * (1 - \alpha) \\ &= \{ \text{step}_6 \} = \\ &\text{ev} [(r_0^0 + r_1^{0,0} * \beta + r_1^{0,1} * (1 - \beta), \alpha), (r_0^1 + r_1^{1,0}, 1 - \alpha)] \\ &= \{ \text{step}_7 \} = \\ &\text{ev} [(r_0^0 + \text{ev} [(r_1^{0,0}, \beta), (r_1^{0,1}, 1 - \beta)], \alpha), (r_0^1 + \text{ev} [(r_1^{1,0}, 1)], 1 - \alpha)] \\ &= \{ \text{step}_8 \} = \\ &\text{ev} [(r_0^0 + \text{ev} (\text{fmap}_{SP} \text{sumR} [(((x_1^0, y_1^0) \parallel \text{Last } x_2^{0,0}), \beta), (((x_1^0, y_1^0) \parallel \text{Last } x_2^{0,1}), 1 - \beta)]), \alpha), \\ &\quad (r_0^1 + \text{ev} (\text{fmap}_{SP} \text{sumR} [(((x_1^1, y_1^1) \parallel \text{Last } x_2^{1,0}), 1)]), 1 - \alpha)] \\ &= \{ \text{step}_9 \} = \\ &\text{ev} [(r_0^0 + \text{ev} (\text{fmap}_{SP} \text{sumR} (\text{trj} [p_1] x_1^0)), \alpha), (r_0^1 + \text{ev} (\text{fmap}_{SP} \text{sumR} (\text{trj} [p_1] x_1^1)), 1 - \alpha)] \\ &= \{ \text{step}_{10} \} = \\ &\text{ev} [(\text{reward } 0 \ x_0 \ y_0 \ x_1^0 + \text{val} [p_1] x_1^0, \alpha), (\text{reward } 0 \ x_0 \ y_0 \ x_1^1 + \text{val} [p_1] x_1^1, 1 - \alpha)] \end{aligned}$$

Solution: Bellman's equation

$$\begin{aligned} &= \{ \text{step}_{11} \} = \\ &\text{ev} [(\text{reward } 0 \ x_0 \ y_0 \ x_1^0 + \text{val } [p_1] \ x_1^0, \alpha), (\text{reward } 0 \ x_0 \ y_0 \ x_1^1 + \text{val } [p_1] \ x_1^1, 1 - \alpha)] \\ &= \{ \text{step}_{12} \} = \\ &\text{ev} (\text{fmap}_{SP} (\text{reward } 0 \ x_0 \ y_0 \oplus_I \text{val } [p_1]) [(x_1^0, \alpha), (x_1^1, 1 - \alpha)]) \\ &= \{ \text{step}_{13} \} = \\ &\text{ev} (\text{fmap}_{SP} (\text{reward } 0 \ x_0 \ y_0 \oplus_I \text{val } [p_1]) (\text{next } 0 \ x_0 \ y_0)) \end{aligned}$$

Solution: Bellman's equation

$$\begin{aligned} &= \{ \text{step}_{11} \} = \\ &\text{ev} [(\text{reward } 0 \ x_0 \ y_0 \ x_1^0 + \text{val } [p_1] \ x_1^0, \alpha), (\text{reward } 0 \ x_0 \ y_0 \ x_1^1 + \text{val } [p_1] \ x_1^1, 1 - \alpha)] \\ &= \{ \text{step}_{12} \} = \\ &\text{ev} (\text{fmap}_{SP} (\text{reward } 0 \ x_0 \ y_0 \oplus_I \text{val } [p_1]) [(x_1^0, \alpha), (x_1^1, 1 - \alpha)]) \\ &= \{ \text{step}_{13} \} = \\ &\text{ev} (\text{fmap}_{SP} (\text{reward } 0 \ x_0 \ y_0 \oplus_I \text{val } [p_1]) (\text{next } 0 \ x_0 \ y_0)) \end{aligned}$$

Thus, we have computed

$$\text{val } [p_0, p_1] \ x_0 = \text{ev} (\text{fmap}_{SP} (\text{reward } 0 \ x_0 \ y_0 \oplus_I \text{val } [p_1]) (\text{next } 0 \ x_0 \ y_0))$$

Solution: Bellman's equation

Exercise 7.5

Is the computation correct? Check it and report any errors you find!

Exercise 7.6

Redo the computation for the non-deterministic case with the canonical monadic operations for *List* and with *measure* = *sum*. Do you obtain the same computational pattern?

Solution: Bellman's equation

Exercise 7.5

Is the computation correct? Check it and report any errors you find!

Exercise 7.6

Redo the computation for the non-deterministic case with the canonical monadic operations for *List* and with *measure* = *sum*. Do you obtain the same computational pattern?

- For stochastic SDPs, one can generalize the result to

$$\text{val } (p :: ps) \ x = \text{ev } (\text{fmap}_{SP} (\text{reward } t \ x \ (p \ x) \oplus_I \ \text{val } ps) (\text{next } t \ x \ (p \ x)))$$

for arbitrary *p*, *ps*, *reward* and *next* of consistent types.

Solution: Bellman's equation

Exercise 7.5

Is the computation correct? Check it and report any errors you find!

Exercise 7.6

Redo the computation for the non-deterministic case with the canonical monadic operations for *List* and with *measure* = *sum*. Do you obtain the same computational pattern?

- For stochastic SDPs, one can generalize the result to

$$\text{val } (p :: ps) \ x = \text{ev } (\text{fmap}_{SP} (\text{reward } t \ x \ (p \ x) \oplus_I \ \text{val } ps) (\text{next } t \ x \ (p \ x)))$$

for arbitrary *p*, *ps*, *reward* and *next* of consistent types. This is *Bellman's equation*!

Solution: Bellman's equation

Solution: Bellman's equation

Exercise 7.7

Not surprisingly, Bellman's equation also holds for the "plain" deterministic case. Prove

$$\text{BellmanEq} : (t\ n : \mathbb{N}) \rightarrow (p : \text{Policy } t) \rightarrow (ps : \text{PolicySeq } (\text{suc } t)\ n) \rightarrow (x : X\ t) \rightarrow \\ \text{val } (p :: ps)\ x \equiv \text{measure } (\text{fmap } (\text{reward } t\ x\ (p\ x)) \oplus_I \text{val } ps) (\text{next } t\ x\ (p\ x)))$$

by induction on ps and with $M = \text{Id}$, $\text{Id } X = X$, $\text{measure} = \text{id}$, arbitrary next ,

$$\text{val } ps = \text{measure} \circ \text{fmap } \text{sumR} \circ \text{trj } ps$$

and with $\text{fmap}_{\text{Id}} = \eta_{\text{Id}} = \text{id}$ and $x \gg=_{\text{Id}} f = f\ x$ for the identity monad. Apply

$$\text{Lemma} : (t\ n : \mathbb{N}) \rightarrow (p : \text{Policy } t) \rightarrow (ps : \text{PolicySeq } (\text{suc } t)\ n) \rightarrow (x : X\ t) \rightarrow \\ \text{sumR } (\text{trj } (p :: ps)\ x) \equiv \text{reward } t\ x\ (p\ x) (\text{next } t\ x\ (p\ x)) \oplus \text{val } ps (\text{next } t\ x\ (p\ x))$$

Solution: Bellman's equation

- If *measure*, $\oplus$ and M fulfill certain *compatibility conditions*, Bellman's equation can be generalized to the *monadic* case.

Solution: Bellman's equation

- If *measure*, $\oplus$ and *M* fulfill certain *compatibility conditions*, Bellman's equation can be generalized to the monadic case.
- In this case one defines the **value** of a **policy sequence** through Bellman's equation:

$$\begin{aligned} \text{val} &: \{t\ n : \mathbb{N}\} \rightarrow \text{PolicySeq } t\ n \rightarrow X\ t \rightarrow \text{Val} \\ \text{val } \{t\} \text{ Nil } x &= 0_{\text{Val}} \\ \text{val } \{t\} (p :: ps) x &= \text{let } y = p\ x \text{ in} \\ &\quad \text{let } mx' = \text{next } t\ x\ y \text{ in} \\ &\quad \text{measure } (\text{fmap}_M (\text{reward } t\ x\ y \oplus_I \text{val } ps) mx') \end{aligned}$$

Solution: Bellman's equation

- If *measure*, $\oplus$ and *M* fulfill certain *compatibility conditions*, Bellman's equation can be generalized to the monadic case.
- In this case one defines the value of a policy sequence through Bellman's equation:

$$\begin{aligned} \text{val} &: \{t\ n : \mathbb{N}\} \rightarrow \text{PolicySeq } t\ n \rightarrow X\ t \rightarrow \text{Val} \\ \text{val } \{t\} \text{ Nil } x &= 0_{\text{Val}} \\ \text{val } \{t\} (p :: ps) x &= \text{let } y = p\ x \text{ in} \\ &\quad \text{let } mx' = \text{next } t\ x\ y \text{ in} \\ &\quad \text{measure } (\text{fmap}_M (\text{reward } t\ x\ y \oplus_I \text{val } ps) mx') \end{aligned}$$

- Note the difference between **this** computation and the **original** computation of *val*:

$$\begin{aligned} \text{val} &: \{t\ n : \mathbb{N}\} \rightarrow \text{PolicySeq } t\ n \rightarrow X\ t \rightarrow \text{Val} \\ \text{val } ps\ x &= \text{measure } (\text{fmap}_M \text{sumR } (\text{trj } ps\ x)) \end{aligned}$$

Solution: Bellman's principle, optimal extensions

- Bellman's equation is the key for solving SDPs via backward induction.

Solution: Bellman's principle, optimal extensions

- Bellman's equation is the key for solving SDPs via backward induction.
- Backward induction follows directly from Bellman's optimality principle:

Solution: Bellman's principle, optimal extensions

- Bellman's equation is the key for solving SDPs via backward induction.
- Backward induction follows directly from Bellman's optimality principle:
- Optimal extensions of optimal policy sequences are optimal.

Solution: Bellman's principle, optimal extensions

- Bellman's equation is the key for solving SDPs via backward induction.
- Backward induction follows directly from Bellman's optimality principle:
- Optimal extensions of optimal policy sequences are optimal.

$$\begin{aligned} \text{Bellman} : \{t\ n : \mathbb{N}\} \rightarrow (p : \text{Policy } t) \rightarrow (ps : \text{PolicySeq } (\text{suc } t)\ n) \rightarrow \\ \text{OptExt } ps\ p \rightarrow \text{OptPolicySeq } ps \rightarrow \text{OptPolicySeq } (p :: ps) \end{aligned}$$

Solution: Bellman's principle, optimal extensions

- Bellman's equation is the key for solving SDPs via backward induction.
- Backward induction follows directly from Bellman's optimality principle:
- Optimal extensions of optimal policy sequences are optimal.

$$\begin{aligned} \text{Bellman} : \{ t \ n : \mathbb{N} \} &\rightarrow (p : \text{Policy } t) \rightarrow (ps : \text{PolicySeq } (\text{suc } t) \ n) \rightarrow \\ &\text{OptExt } ps \ p \rightarrow \text{OptPolicySeq } ps \rightarrow \text{OptPolicySeq } (p :: ps) \end{aligned}$$

- p is an **optimal extension** of ps iff $p :: ps$ is at least as good as any other extension $p' :: ps$:

Solution: Bellman's principle, optimal extensions

- Bellman's equation is the key for solving SDPs via backward induction.
- Backward induction follows directly from Bellman's optimality principle:
- Optimal extensions of optimal policy sequences are optimal.

$$\text{Bellman} : \{t \ n : \mathbb{N}\} \rightarrow (p : \text{Policy } t) \rightarrow (ps : \text{PolicySeq } (\text{succ } t) \ n) \rightarrow \\ \text{OptExt } ps \ p \rightarrow \text{OptPolicySeq } ps \rightarrow \text{OptPolicySeq } (p :: ps)$$

- p is an **optimal extension** of ps iff $p :: ps$ is at least as good as any other extension $p' :: ps$:

$$\text{OptExt} : \{t \ n : \mathbb{N}\} \rightarrow \text{PolicySeq } (\text{succ } t) \ n \rightarrow \text{Policy } t \rightarrow \text{Set} \\ \text{OptExt } ps \ p = \forall p' \rightarrow \text{val } (p' :: ps) \leq_I \text{val } (p :: ps)$$

Solution: Bellman's principle, optimal extensions

Exercise 7.8

Thus, computing an optimal extension $p : \text{Policy } t$ of a policy sequence $ps : \text{PolicySeq } (\text{succ } t) \ n$ requires computing a control $p \ x : Y \ t \ x$ that maximizes $\text{val } (p :: ps) \ x$ for every $x : X \ t$. When $Y \ t \ x$ is finite and non-empty, this can be done by **linear search**. Postulate

$$\begin{aligned} \text{Finite NonEmpty} &: \text{Set} \rightarrow \text{Set} \\ \text{NonEmptyList} &: \{A : \text{Set}\} \rightarrow \text{List } A \rightarrow \text{Set} \\ \text{toList} &: \{A : \text{Set}\} \rightarrow \text{Finite } A \rightarrow \text{NonEmpty } A \rightarrow \Sigma (\text{List } A) \text{ NonEmptyList} \\ \text{argmax} &: \{A : \text{Set}\} \rightarrow (f : A \rightarrow \text{Val}) \rightarrow \Sigma (\text{List } A) \text{ NonEmptyList} \rightarrow A \\ fY &: \{t : \mathbb{N}\} \rightarrow (x : X \ t) \rightarrow \text{Finite } (Y \ t \ x) \\ \text{neY} &: \{t : \mathbb{N}\} \rightarrow (x : X \ t) \rightarrow \text{NonEmpty } (Y \ t \ x) \end{aligned}$$

and then implement

$$\text{optExt} : \{t \ n : \mathbb{N}\} \rightarrow \text{PolicySeq } (\text{succ } t) \ n \rightarrow \text{Policy } t$$

Solution: Bellman's principle, optimal extensions

Exercise 7.9

Formulate **minimal** requirements on *toList* and *argmax* for *optExt* to satisfy

$$\text{optExtSpec} : \{t\ n : \mathbb{N}\} \rightarrow (ps : \text{PolicySeq}\ (suc\ t)\ n) \rightarrow \text{OptExt}\ ps\ (\text{optExt}\ ps)$$

Solution: Bellman's principle, optimal extensions

Exercise 7.9

Formulate minimal requirements on *toList* and *argmax* for *optExt* to satisfy

$$\text{optExtSpec} : \{t\ n : \mathbb{N}\} \rightarrow (ps : \text{PolicySeq}\ (suc\ t)\ n) \rightarrow \text{OptExt}\ ps\ (\text{optExt}\ ps)$$

- To prove Bellman's optimality principle one needs two **monotonicity conditions**

Solution: Bellman's principle, optimal extensions

Exercise 7.9

Formulate minimal requirements on *toList* and *argmax* for *optExt* to satisfy

$$\text{optExtSpec} : \{t\ n : \mathbb{N}\} \rightarrow (ps : \text{PolicySeq}\ (suc\ t)\ n) \rightarrow \text{OptExt}\ ps\ (\text{optExt}\ ps)$$

- To prove Bellman's optimality principle one needs two **monotonicity conditions**

$$\begin{aligned} \text{measureMon} : \{A : \text{Set}\} \rightarrow (f\ g : A \rightarrow \text{Val}) \rightarrow (f \leq_I g) \rightarrow \\ (\text{measure} \circ \text{fmap}_M\ f) \leq_I (\text{measure} \circ \text{fmap}_M\ g) \end{aligned}$$

Solution: Bellman's principle, optimal extensions

Exercise 7.9

Formulate minimal requirements on *toList* and *argmax* for *optExt* to satisfy

$$\text{optExtSpec} : \{t\ n : \mathbb{N}\} \rightarrow (ps : \text{PolicySeq}\ (suc\ t)\ n) \rightarrow \text{OptExt}\ ps\ (\text{optExt}\ ps)$$

- To prove Bellman's optimality principle one needs two **monotonicity conditions**

$$\begin{aligned} \text{measureMon} : \{A : \text{Set}\} \rightarrow (f\ g : A \rightarrow \text{Val}) \rightarrow (f \leq_I g) \rightarrow \\ (\text{measure} \circ \text{fmap}_M f) \leq_I (\text{measure} \circ \text{fmap}_M g) \end{aligned}$$

$$\text{plusMon} : \{a\ b\ c\ d : \text{Val}\} \rightarrow a \leq b \rightarrow c \leq d \rightarrow (a \oplus c) \leq (b \oplus d)$$

Solution: Bellman's principle, optimal extensions

Exercise 7.9

Formulate minimal requirements on *toList* and *argmax* for *optExt* to satisfy

$$\text{optExtSpec} : \{t\ n : \mathbb{N}\} \rightarrow (ps : \text{PolicySeq}\ (suc\ t)\ n) \rightarrow \text{OptExt}\ ps\ (\text{optExt}\ ps)$$

- To prove Bellman's optimality principle one needs two monotonicity conditions

$$\begin{aligned} \text{measureMon} : \{A : \text{Set}\} &\rightarrow (f\ g : A \rightarrow \text{Val}) \rightarrow (f \leq_I g) \rightarrow \\ &(\text{measure} \circ \text{fmap}_M\ f) \leq_I (\text{measure} \circ \text{fmap}_M\ g) \end{aligned}$$

$$\text{plusMon} : \{a\ b\ c\ d : \text{Val}\} \rightarrow a \leq b \rightarrow c \leq d \rightarrow (a \oplus c) \leq (b \oplus d)$$

Exercise 7.10

Postulate *measureMon*, *plusMon* and implement *Bellman*.

Solution: verified backward induction

- With *optExt* one can solve SDPs by backward induction:

Solution: verified backward induction

- With *optExt* one can solve SDPs by backward induction:

$$bi : (t\ n : \mathbb{N}) \rightarrow PolicySeq\ t\ n$$
$$bi\ t\ zero = Nil$$
$$bi\ t\ (suc\ n) = \mathbf{let}\ ps = bi\ (suc\ t)\ n\ \mathbf{in}\ optExt\ ps :: ps$$

Solution: verified backward induction

- With *optExt* one can solve SDPs by backward induction:

$$bi : (t\ n : \mathbb{N}) \rightarrow PolicySeq\ t\ n$$
$$bi\ t\ zero = Nil$$
$$bi\ t\ (suc\ n) = \mathbf{let}\ ps = bi\ (suc\ t)\ n\ \mathbf{in}\ optExt\ ps :: ps$$

- With *Bellman* and *optExtSpec* one can **verify** that *bi* yields **optimal** policy sequences

$$biOptVal : (t\ n : \mathbb{N}) \rightarrow OptPolicySeq\ (bi\ t\ n)$$

Solution: verified backward induction

Exercise 7.11

Implement *biOptVal* by induction on n

$$biOptVal\ t\ zero =$$

$$biOptVal\ t\ (suc\ n) =$$

Notice that policy sequences for zero decision steps are optimal by reflexivity of $\leq$.

- This was an account of the simplified theory presented in the *MatterMost* paper.

Verified policy advice: wrap-up, technical issues

- This was an account of the simplified theory presented in the *MatterMost* paper.
- The bottom line is that, if M , $\leq$ and $\oplus$ fulfil fairly natural conditions, one can compute verified optimal policies for arbitrary finite-horizon SDPs.

Verified policy advice: wrap-up, technical issues

- This was an account of the simplified theory presented in the *MatterMost* paper.
- The bottom line is that, if M , $\leq$ and $\oplus$ fulfil fairly natural conditions, one can compute verified optimal policies for arbitrary finite-horizon SDPs.
- The simplified theory is easy to discuss but has a major flaw: what if a control set Y_{tx} is empty? Or if *next* returns an empty M -structure of possible next states?

Verified policy advice: wrap-up, technical issues

- This was an account of the simplified theory presented in the *MatterMost* paper.
- The bottom line is that, if M , $\leq$ and $\oplus$ fulfil fairly natural conditions, one can compute verified optimal policies for arbitrary finite-horizon SDPs.
- The simplified theory is easy to discuss but has a major flaw: what if a control set $Y \ t \ x$ is empty? Or if *next* returns an empty M -structure of possible next states?
- We can “fix” the theory by requiring $Y \ t \ x$, $next \ t \ x \ y$ to contain at least one element or ...

Verified policy advice: wrap-up, technical issues

- This was an account of the simplified theory presented in the *MatterMost* paper.
- The bottom line is that, if M , $\leq$ and $\oplus$ fulfil fairly natural conditions, one can compute verified optimal policies for arbitrary finite-horizon SDPs.
- The simplified theory is easy to discuss but has a major flaw: what if a control set $Y \ t \ x$ is empty? Or if *next* returns an empty M -structure of possible next states?
- We can “fix” the theory by requiring $Y \ t \ x$, $next \ t \ x \ y$ to contain at least one element or ...
- ... by building a more general theory, as done in *JFP2017* by **restricting** the domain of policies to states that are **viable** for a suitable number of decision steps.

Verified policy advice: limitations

- The theory is general enough to support verified optimal decision making under uncertainty for a finite number of decision steps

Verified policy advice: limitations

- The theory is general enough to support verified optimal decision making under uncertainty for a finite number of decision steps
- What about optimal decision making for infinitely many decision steps?

Verified policy advice: limitations

- The theory is general enough to support verified optimal decision making under uncertainty for a finite number of decision steps
- What about optimal decision making for infinitely many decision steps?
- What if one is required to provide decision makers with **all optimal** policy sequences rather than just one?

Verified policy advice: limitations

- The theory is general enough to support verified optimal decision making under uncertainty for a finite number of decision steps
- What about optimal decision making for infinitely many decision steps?
- What if one is required to provide decision makers with all optimal policy sequences rather than just one?
- There are also more “practical” limitations . . .

Verified policy advice: limitations

- The theory is general enough to support verified optimal decision making under uncertainty for a finite number of decision steps
- What about optimal decision making for infinitely many decision steps?
- What if one is required to provide decision makers with all optimal policy sequences rather than just one?
- There are also more “practical” limitations ...

Exercise 7.12

... which ones come to mind?